

NEXUS AI

Implementation Plan

Primary Problem Statement: Demand Forecasting and Order Fulfillment

Delivery Constraint: 8-hour hackathon MVP

Current Status: Product scope, PRD/TRP, application flow and architecture are defined; implementation is ready to begin.

Implementation Principle: Ship one reliable end-to-end business workflow before adding secondary features.

Data → Forecast → Risk → Recommendation → Vendor → Order → Fulfillment → Alert

1. Executive Summary

NEXUS AI is an AI-powered decision-support platform for demand forecasting and order fulfillment. The implementation target is a working hackathon MVP that demonstrates a complete business journey rather than a large number of partially implemented modules.

The 8-hour plan deliberately prioritizes the core workflow: ingest business data, forecast demand, identify inventory risk, recommend replenishment and a supplier, track fulfillment, and surface actionable alerts. The AI Assistant is added only after the core workflow is stable.

Scope Lock

- Must-have: data pipeline, forecasting, inventory risk, purchase recommendation, vendor recommendation, fulfillment tracking, command-center dashboard.
- Secondary: grounded AI Assistant, what-if simulation if time remains.
- Deferred: full ERP, invoice OCR, complex approval engine, mobile app, autonomous purchasing, multi-tenant architecture, microservices.

Assumptions

- Timeline: 8 hours from implementation start to deployment/demo.
- Team composition: [Insert actual team size/roles]. Workstreams should run in parallel where possible.
- Tech stack: React/Next.js, Tailwind CSS, Python/FastAPI, PostgreSQL, Pandas/NumPy/scikit-learn/XGBoost, and [LLM Provider].
- A clean hackathon dataset may be created or prepared rather than spending excessive time sourcing a perfect external dataset.

2. Delivery Strategy & Timeline

Time	Phase	Primary Outcome
0:00–0:30	Phase 1	Repository, environments, schema and development contract locked
0:30–1:30	Phase 2A	Dataset, database and data pipeline ready
1:30–3:00	Phase 2B	Forecasting + risk engine working
3:00–4:00	Phase 2C	Purchase + vendor recommendation working
4:00–5:30	Phase 2D	Backend APIs integrated
5:30–7:00	Phase 3	Dashboard and core UI working
7:00–7:30	Phase 4	Grounded AI Assistant + QA
7:30–8:00	Phase 5	Deployment, smoke test and demo lock

3. Phase 1 — Environment & Repository Setup

Duration: 0:00–0:30

Objective: Establish a shared, reproducible development environment and freeze the MVP contract before implementation.

Repository Structure

```
nexus-ai/  
■■■ frontend/  
■■■ backend/  
■■■ ml/  
■■■ data/
```

```
■■■ docs/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

Actionable Deliverables

- Create the Git repository and protected main branch.
- Create frontend, backend, ML, data and docs directories.
- Create .gitignore and .env.example.
- Define local run commands for frontend and backend.
- Define the shared API/data contract before parallel development.
- Initialize PostgreSQL database or selected managed development database.
- Create a seed/demo dataset plan.
- Set up basic CI checks: lint/build/test where practical.
- Create a short task board with owner + status for every MVP deliverable.

Exit Criteria

- Every developer can clone and run the project.
- No secrets are committed.
- Repository structure is stable.
- MVP scope is frozen.
- Frontend, backend and database can communicate in the development environment.

4. Phase 2 — Backend, Data & AI/ML Development

Duration: 0:30–5:30, with parallel workstreams.

Objective: Build the functional intelligence and backend contract that powers the end-to-end business journey.

4.1 Data & Database

Create the minimum data model required by the MVP.

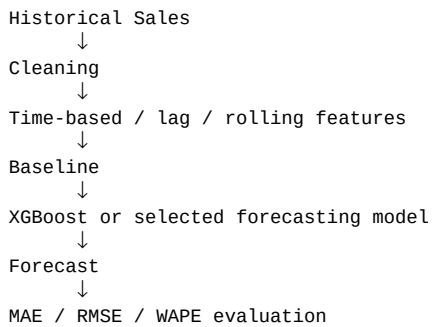
```
Products
Sales
Inventory
Suppliers
Orders
Shipments
Forecasts
Risk Events
Recommendations
Alerts
Audit Events
```

Deliverables

- Create database tables and indexes.
- Create migration/initialization scripts.
- Load realistic demo data.
- Validate dates, product IDs, quantities and required fields.
- Create reusable data-loading and preprocessing functions.
- Ensure the dataset supports at least one obvious high-risk scenario for the final demo.

4.2 Demand Forecasting

Build the forecasting model without overengineering. Establish a simple baseline and compare the main model against it.

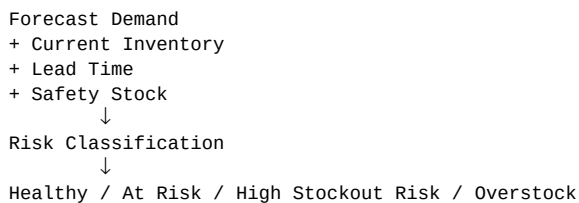


Deliverables

- Create preprocessing pipeline.
- Create train/test split appropriate for time-series data.
- Implement baseline.
- Implement selected forecasting model.
- Calculate actual evaluation metrics.
- Store forecast output with product, horizon, value and model version.
- Expose forecast generation through a backend service/API.

4.3 Inventory Risk Engine

Keep risk logic deterministic and explainable.



Deliverables

- Implement risk calculation.
- Create clear thresholds/configuration.
- Return risk status and reason codes.
- Return human-readable explanation factors.
- Expose risk results through an API.

4.4 Recommendation & Vendor Engine

Separate prediction from decision. The ML model predicts demand; deterministic business logic decides the replenishment action.

Deliverables

- Calculate recommended purchase quantity.
- Implement supplier scoring.
- Use price, delivery, reliability and other available supplier metrics.
- Return ranked suppliers.

- Return recommendation rationale.
- Expose recommendations through an API.
- Keep human approval in the loop for consequential actions.

4.5 Backend API

```
/api/products
/api/sales
/api/inventory
/api/forecast
/api/risks
/api/recommendations
/api/suppliers
/api/orders
/api/shipments
/api/alerts
/api/assistant
/api/audit
```

Deliverables

- Create FastAPI routes and service layer.
- Validate request and response schemas.
- Connect APIs to PostgreSQL and ML services.
- Add structured error responses.
- Add CORS restrictions for configured frontend origins.
- Add basic rate limiting where practical.
- Implement server-side authorization boundaries.
- Log important actions for auditability.

5. Phase 3 — Frontend Development

Duration: 5:30–7:00

Objective: Build a focused command center that makes the business story obvious within seconds.

UI Priorities

- Command Center
- Forecast chart
- Inventory risk cards/table
- Purchase recommendations
- Supplier ranking
- Order/fulfillment status
- Critical alerts
- AI Assistant panel

Deliverables

- Initialize UI component system and layout.
- Implement responsive command-center dashboard.
- Connect dashboard to backend APIs.
- Add loading, empty and error states.

- Show forecast versus historical demand.
- Show risk status with clear explanations.
- Show purchase quantity and supplier recommendation.
- Show order lifecycle and delayed shipments.
- Add drill-down from alert → product/order → explanation.
- Avoid adding decorative pages that do not support the demo flow.

Frontend State Strategy

Use a simple server-state/data-fetching approach appropriate to the selected frontend stack. Keep API data, UI state and derived visualization state separate. Do not duplicate business logic in the frontend.

6. Phase 4 — Security, QA & Testing

Duration: 7:00–7:30, with testing occurring continuously throughout development.

Objective: Validate the core workflow, security boundaries and failure behavior before deployment.

Testing Layers

- Unit tests: risk calculation, purchase calculation, vendor scoring and key data transformations.
- API tests: validation, success responses, authorization and error handling.
- ML tests: preprocessing consistency and model evaluation.
- Integration tests: API → database → ML/risk/recommendation pipeline.
- End-to-end test: forecast → risk → recommendation → supplier → order → fulfillment → alert.
- Security checks: dependency scan, secret scan, CORS verification and authorization tests.

Security Deliverables

- Verify environment variables and secret handling.
- Verify server-side authorization.
- Validate API input.
- Restrict CORS.
- Apply rate limits to expensive/AI endpoints where feasible.
- Ensure error messages do not leak stack traces or secrets.
- Verify audit events for consequential actions.

QA Sign-off Criteria

- Golden demo scenario completes without manual database manipulation.
- No critical runtime errors.
- Forecast and risk values are generated from actual implementation logic.
- Recommendation explanations match the underlying calculations.
- Delayed fulfillment is detected correctly.
- AI Assistant answers are grounded in current application data.
- No secrets are present in the repository or frontend bundle.

7. Phase 5 — Deployment & Go-Live

Duration: 7:30–8:00

Objective: Deploy the stable MVP and lock the exact demo path.

Deployment Flow

```
graph TD
    A[Git Commit] --> B[Build / Test]
    B --> C[Staging or Preview]
    C --> D[Smoke Test]
    D --> E[Production]
    E --> F[Final Demo Verification]
```

Deliverables

- Provision frontend hosting.
- Provision backend hosting.
- Provision managed PostgreSQL or configured production database.
- Configure production environment variables/secrets.
- Apply database initialization/migrations.
- Load approved demo data.
- Run production smoke tests.
- Verify API connectivity and frontend rendering.
- Verify AI provider configuration if used.
- Capture final deployment URL.
- Freeze the demo branch/tag.
- Prepare the 2–4 minute final demo flow.

Go-Live Checklist

- Application opens successfully.
- Dashboard data loads.
- Forecast chart works.
- High-risk product is visible.
- Recommendation is generated.
- Supplier ranking works.
- Order status works.
- Delayed shipment alert works.
- AI Assistant can answer the planned demo questions.
- No obvious console/runtime errors.
- Demo data is deterministic and repeatable.

8. Visual Timeline — Mermaid.js Gantt

```
gantt
    title NEXUS AI — 8 Hour Hackathon Implementation
    dateFormat HH:mm
    axisFormat %H:%M
```

```

section Setup
Repository & environment      :p1, 00:00, 30m
Database initialization       :p1b, after p1, 30m

section Backend / AI
Dataset & preprocessing      :p2a, 00:30, 1h
Forecasting model            :p2b, 01:30, 1h
Inventory risk engine        :p2c, after p2b, 30m
Recommendation + vendor engine :p2d, 03:00, 1h
Backend APIs                 :p2e, 03:30, 2h

section Frontend
Dashboard shell              :p3a, 03:30, 1h
API integration              :p3b, 05:00, 1h
Final UI / states            :p3c, 06:00, 1h

section QA / AI
Grounded AI Assistant        :p4a, 07:00, 20m
QA + security smoke tests    :p4b, 07:00, 30m

section Deployment
Production deployment        :p5a, 07:30, 20m
Demo verification            :p5b, 07:50, 10m

```

9. Risk Management & Mitigations

Risk	Impact	Mitigation
Scope creep	Critical	Freeze MVP scope. No secondary feature is allowed to block the core workflow.
ML model takes too long	High	Start with a baseline and a simple validated model. Use the best model that can be evaluated quickly.
Frontend/backend integration delays	High	Define API contracts early and use mock responses while services are developed in parallel.
Deployment failure near deadline	Critical	Deploy a basic working version early; keep production configuration documented and perform a smoo

Additional Technical Risks

- Poor demo data: mitigate by designing at least one deterministic high-risk scenario.
- LLM failure: keep the core dashboard and recommendation workflow independent of the LLM.
- Incorrect recommendation: expose the calculation factors and keep consequential actions human-approved.
- Security oversights: run secret scanning and dependency/security checks before final deployment.

10. Agile Execution Model

The team should work in short implementation loops rather than completing entire layers sequentially.

PLAN → BUILD → INTEGRATE → TEST → DEMO CHECK → IMPROVE

Parallel Workstreams

Workstream	Primary Ownership	Outputs
Data / ML	[Team Member]	Dataset, preprocessing, forecast, metrics
Backend	[Team Member]	DB, APIs, risk, recommendation, fulfillment
Frontend	[Team Member]	Dashboard, charts, tables, interactions
Integration / AI / DevOps	[Team Member]	AI Assistant, deployment, QA, integration

If the team is smaller, combine roles. If larger, split ownership by feature rather than creating duplicate implementations.

11. Definition of Done

- Code is committed and runnable from a clean clone.

- Feature is integrated with the main application.
- Relevant unit/integration tests pass.
- Error and loading states are handled.
- No secrets are committed.
- Business logic is implemented on the backend rather than duplicated in the UI.
- The feature contributes to the core NEXUS journey.
- The feature is demonstrated using realistic data.
- The team can explain how it works and why the result is trustworthy.

12. Final Delivery Objective

At the end of the 8-hour window, NEXUS AI should not attempt to look like a complete ERP. It should look like a focused, technically credible decision-intelligence product that solves one business problem end-to-end.



Success condition: A judge can enter the dashboard, see a problem, inspect why it exists, see what NEXUS recommends, follow the resulting order, and understand how the system turns data into an actionable business decision.